

Património 360 — Assistente Virtual de Museus

Documento de Arquitetura Completa e Funcionalidades

Documento técnico **exaustivo**. Descreve a arquitetura completa da aplicação, todas as funcionalidades, o pipeline de raciocínio (RAG multimodal), os mecanismos de comunicação, o modelo de dados, a observabilidade e **exemplos de todos os tipos de queries** suportados. Tudo o que aqui está descrito foi verificado diretamente no código-fonte do repositório.

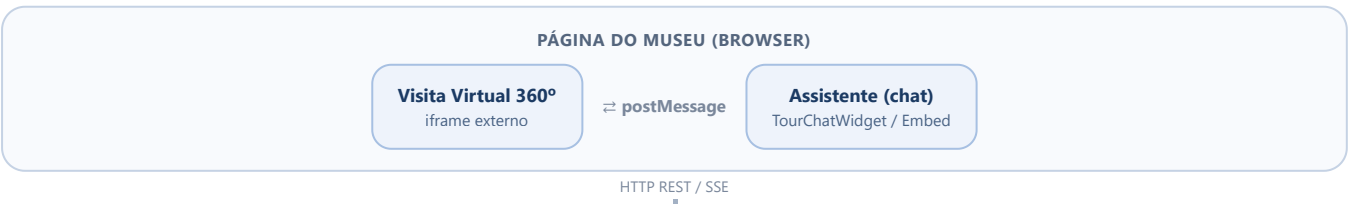
Índice

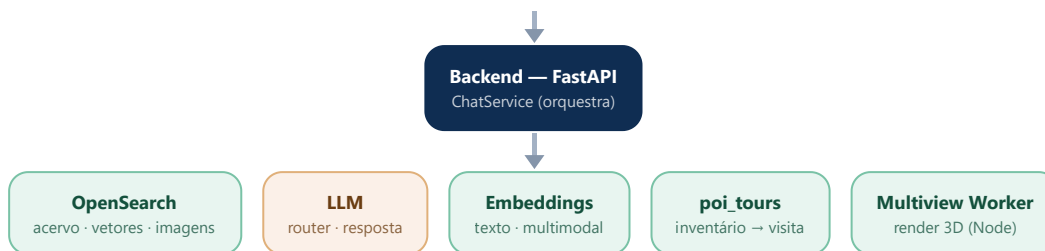
1. Visão geral
2. Stack tecnológica
3. Mapa de componentes
4. O pipeline de raciocínio (passo a passo)
5. O Router: como a pergunta é encaminhada
6. Política de contexto e seguimento (follow-up)
7. Retrieval de texto (pesquisa híbrida)
8. Filtros: museu, temporais e âmbito da visita
9. Reescrita de query e *boosts*
10. Entradas multimodais (texto, imagem, modelo 3D)
11. Navegação peça → ponto da visita virtual
12. Geração da resposta final e sanitização
13. Paginação de resultados
14. Modal de detalhe: autores, conjuntos, exposições, relacionados
15. Pesquisa entre museus (*cross-museum*)
16. Comunicação Frontend ↔ Backend (REST + SSE)
17. Comunicação Frontend ↔ Visita Virtual (postMessage)
18. Sessão, estado e memória de conversa
19. Internacionalização (PT/EN)
20. Observabilidade: logging estruturado de queries e interações
21. Configuração (settings/env)
22. Modelo de dados (artefacto e entidades)
23. **Exemplos de todos os tipos de queries**
24. Apêndice A — Esquema do Router (JSON)
25. Apêndice B — Esquema do evento `backend_query`
26. Apêndice C — Notas de implementação e componentes não ativos

1. Visão geral

O projeto é um **assistente conversacional embebido numa visita virtual 360° de um museu**. O visitante faz perguntas — por **texto, imagem ou modelo 3D** — e o sistema procura nas peças do acervo, responde em linguagem natural com base em factos recuperados e, quando faz sentido, **leva o visitante até ao ponto exato da visita virtual** onde a peça está exposta.

Em uma frase: *é um guia inteligente para uma visita virtual de museu, que percebe linguagem natural, pesquisa multimodal no acervo e liga cada peça ao seu lugar físico na visita 360°.*





Peças-chave:

- **Frontend (React/Vite)** — widget de chat sobreposto à visita virtual; é "fino": trata da UI e da ponte para a visita, mas não contém lógica de raciocínio.
- **Backend (FastAPI / ChatService)** — orquestra todo o raciocínio e detém o estado.
- **OpenSearch** — base de pesquisa do acervo, com três "vistas" do mesmo dado: texto pesquisável (BM25), vetores semânticos de texto e vetores multimodais de imagem.
- **LLM** (OpenAI-compatible) — usado para (1) *router* (decidir o caminho), (2) reescrita da query lexical de retrieval, (3) interpretação temporal e (4) redação da resposta final.
- **Embeddings** — modelos de *embedding* de texto e multimodal (imagem).
- **poi_tours/** — ficheiros JSON que ligam o **número de inventário** de uma peça ao **ponto da visita virtual** (panorama + *overlay*).
- **Multiview Worker** — serviço Node/Puppeteer que renderiza várias vistas de um modelo 3D enviado pelo utilizador, para pesquisa por modelo.

2. Stack tecnológica

Camada	Tecnologia
Frontend	React + TypeScript + Vite
Backend	Python 3.11+, FastAPI, Pydantic / pydantic-settings
Cliente LLM	SDK OpenAI (AsyncOpenAI), endpoint <i>openai-compatible</i>
Pesquisa	OpenSearch (índices de artefactos, imagens, museus e entidades relacionais)
Embeddings de texto	BGE-M3 (1024-dim) via FlagEmbedding ou OpenRouter; alternativa Qwen
Embeddings multimodais	Qwen3-VL-Embedding-2B (2048-dim) via sentence-transformers
Render 3D	Worker Node.js + Puppeteer (Chromium headless)
Streaming	Server-Sent Events (SSE) sobre HTTP
Persistência de sessão	Em memória, no processo do backend (com TTL)
Observabilidade	Logging estruturado JSONL (queries de backend + interações de frontend)

LLM por omissão: **carminho/AMALIA-9B-50-DPO** (configurável). Temperatura **0.4** para respostas de texto e **0.0** para saídas JSON (router, reescrita, interpretação temporal).

3. Mapa de componentes

3.1. Backend (`backend/app/`)

app/	
├─ main.py	Cria a app FastAPI + CORS + monta o router
├─ api/	
│ └─ router.py	Agrega: /chat, /logs, health
│ └─ routes/	
│ └─ chat.py	Todos os endpoints de chat, upload e artefactos
│ └─ logs.py	Recebe eventos de interação do frontend
│ └─ health.py	Health checks
├─ core/	
│ └─ config.py	Settings (env) + caminhos resolvidos
├─ schemas/	
│ └─ chat.py	Modelos Pydantic de request/response
├─ prompts/	
│ └─ chat_prompts.py	Prompts do router, lexical e resposta final
│ └─ query_planner_prompts.py	Prompt de reescrita da query de retrieval
└─ services/	
└─ chat_service.py	ORQUESTRADOR central (todo o pipeline)
└─ opensearch_client.py	Gateway de pesquisa (híbrida, imagem, entidades)
└─ embeddings.py	Provedores de embedding (texto + multimodal)
└─ llm_service.py	Cliente LLM openai-compatible
└─ model_retrieval.py	Pesquisa por modelo 3D (orquestra render+embed+search)
└─ multiview_renderer.py	Cliente do worker Node (render de vistas 3D)
└─ tour_navigation.py	Inventário → overlay/panorama (poi_tours)
└─ chat_session_store.py	Estado de sessão em memória (TTL, histórico)
└─ chat_i18n.py	Traduções de estados e erros (PT/EN)
└─ query_logger.py	Logging JSONL (backend_query + frontend_events)
└─ warmup.py	Pré-aquecimento opcional do stack
└─ reranker.py	Reranker pairwise (presente, NÃO ativo – ver Apêndice C)

3.2. Frontend (`frontend/src/embed/`)

embed/	
├─ components/	
│ └─ TourAssistantEmbed.tsx	Contentor: visita (iframe) + assistente + ponte postMessage
│ └─ TourChatWidget.tsx	Widget de chat (UI, mensagens, resultados)
│ └─ MessageMarkdown.tsx	Render do markdown das respostas
│ └─ ModelAttachmentViewer.tsx	Pré-visualização de modelo 3D anexado
├─ services/	
│ └─ chatApi.ts	Cliente do backend (REST + SSE, normalização)
│ └─ tourBridge.ts	postMessage → visita (contexto + navegação)
│ └─ interactionLogger.ts	Eventos de interação → /logs/events
│ └─ embedConfig.ts	Configuração do embed
└─ i18n/index.ts	Traduções da UI
└─ types.ts	Tipos partilhados do frontend

4. O pipeline de raciocínio (passo a passo)

RAG = *Retrieval-Augmented Generation*: o modelo **não responde de cor** — primeiro **procura** factos no acervo e só depois **redige** a resposta com base nesses factos.

A característica central é que **não existe um único caminho**: consoante a pergunta, o sistema escolhe o percurso. O diagrama abaixo mostra o fluxo de uma mensagem de texto.





Cada um destes passos é detalhado nas secções seguintes.

5. O Router: como a pergunta é encaminhada

O **router** é uma chamada LLM que devolve **JSON estrito** (ver Apêndice A). Recebe a mensagem atual, o museu, o estado de filtros/ordenação e — só quando há seguimento — um resumo e excerto do histórico. Decide os campos seguintes:

Campo	Significado
<code>mode</code>	<code>rag</code> (procurar no acervo) ou <code>llm_only</code> (responder só com a conversa)
<code>intent</code>	<code>overview</code> , <code>search</code> , <code>refine</code> ou <code>fallback</code>
<code>is_follow_up</code>	A mensagem é um seguimento de algo já dito?
<code>use_history_for_query</code>	O histórico deve construir a query de pesquisa?
<code>use_history_for_answer</code>	O histórico deve ajudar a redigir a resposta?
<code>carry_filters</code> / <code>carry_sort</code>	Manter filtros/ordenação do turno anterior?
<code>rewritten_query</code>	Reescrita conversacional da pergunta (para o prompt da resposta)
<code>needs_retrieval</code>	Precisa mesmo de ir ao acervo?
<code>filters_delta</code> / <code>sort_delta</code>	Alterações de filtro/ordenação propostas
<code>reason</code>	Justificação (acumula <i>tags</i> de guardrails)

Guardrails determinísticos aplicados sobre a saída do LLM (não se confia cegamente):

- Se `intent` for `search` / `refine`, força-se `mode=rag` e `needs_retrieval=true`.
- Se `needs_retrieval=true`, força-se `mode=rag`.
- Se **não** for seguimento, desligam-se `use_history_for_query`, `carry_filters` e `carry_sort`.
- Se a pergunta contiver expressão de **âmbito da visita** ("nesta visita virtual"), trata-se como **nova pesquisa autónoma** (sem herdar contexto nem `artifact_id`).
- Em caso de falha do LLM no router, usa-se um *fallback* heurístico (`_fallback_router_decision`): saudações/small talk → `llm_only`; caso contrário → `rag`.

Importante: a query usada para **retrieval** é a **mensagem literal do utilizador** (estrita, sem expansões). A `rewritten_query` do router serve apenas o prompt da resposta final, não a pesquisa. A limpeza para BM25 é feita separadamente (secção 9).

6. Política de contexto e seguimento (follow-up)

Antes do router, o `ChatService` deriva uma **política de contexto** determinística (`_derive_context_policy`) que analisa a mensagem com expressões regulares e o estado da sessão. Esta política é uma "dica" forte para o router e também impõe guardrails.

Situação detetada	Resultado
Mensagem vazia / sem contexto anterior	Pergunta autónoma, sem histórico
Pedido explícito de "nova pesquisa", "outro tema", "do zero"	Autónoma (ignora histórico/filtros)
Expressão de âmbito da visita ("nesta visita 360")	Autónoma, ativa filtro de âmbito
Pronomes/referências ("e a anterior?", "só os do século XVIII", "desse autor")	Seguimento: usa histórico para query e resposta , mantém filtros e ordenação
Pedidos de reformulação ("explica melhor", "detalha", "resume")	Seguimento só para resposta (não repesquisa, não herda filtros)
Tudo o resto	Pergunta autónoma por omissão

Isto permite conversas naturais como:

- "Que azulejos há do século XVIII?" → pesquisa nova.
- "E só os de iconografia mariana?" → seguimento que **acrescenta** um filtro à pesquisa anterior.
- "Explica melhor o primeiro." → não repesquisa; reusa o contexto e detalha.

7. Retrieval de texto (pesquisa híbrida)

Quando `mode=rag` e não há número de inventário, executa-se `_retrieve_context`, que faz uma **pesquisa híbrida** no índice de artefactos do OpenSearch, combinando duas vistas:

1. **Semântica (kNN)** — o *embedding* da query é comparado com o campo `text_embedding` dos documentos (`k = 1000` candidatos no kNN).
2. **Lexical (BM25)** — `multi_match` em vários níveis sobre campos do artefacto:

Nível	Tipo	Campos	Boost
Frase	<code>phrase</code> (slop 1)	campos núcleo (título, descrição, categoria, material...)	4.0
Conjunção	<code>best_fields</code> , <code>operator=and</code>	campos núcleo	2.5
Disjunção	<code>best_fields</code> , <code>or</code> , <code>minimum_should_match=2<75%</code>	campos núcleo	1.0
Contexto	<code>best_fields</code> , <code>or</code>	campos de contexto (origem, exposições, bibliografia...)	0.25

As duas vistas são combinadas via *search pipeline* `nlp-search-pipeline` do OpenSearch, com `pagination_depth` igual à janela de recuperação configurada. Pode forçar-se modo **só-embedding** (`CHAT_RETRIEVAL_EMBEDDING_ONLY=true`), que remove os ramos BM25.

Campos núcleo (com pesos): `title^6`, `description^2.5`, `category.text^3`, `super_category.text^2.5`, `support_or_material.text^2`, `technique.text^1.5`, `date_or_period.text^1.2`, `creators.text^1.2`, `production_center.text^0.8`. **Campos de contexto**: `origin_history^0.25`, `exhibitions.text^0.2`, `sets.text^0.2`, `incorporation.text^0.2`, `bibliography^0.1`, `museum.text^0.1`.

O resultado é uma página de documentos já normalizados (cada um com `artifact_id`, `inventory_number`, `title`, `score`, etc.), o total estimado e o **corpo exato da query enviada ao OpenSearch** (guardado para o log — ver secção 20).

8. Filtros: museu, temporais e âmbito da visita

A pesquisa é sempre restringida ao **museu** (cláusula `term` sobre `museum_id`, resolvido a partir do `museum_id` explícito ou do `museum_slug`). Sobre isto aplicam-se filtros adicionais derivados da pergunta:

8.1. Filtro temporal

A pergunta é analisada por `_interpret_temporal_query`, em três camadas:

1. **Explícito por regex** — intervalos como "entre 1700 e 1750", "de 1500 a 1520".
2. ****Períodos históricos *hardcoded***** — aliases conhecidos mapeados para intervalos:

- *período pombalino* → 1750–1777
- *período joanino* → 1706–1750
- *período manuelino* → 1495–1521
- *período sebastianista / crise de sucessão* → 1578–1580
- *período miguelista* → 1828–1834

1. **LLM** — interpretação temporal por modelo quando as duas anteriores não resolvem.

O intervalo resultante vira um **filtro de metadata** (`_temporal_interval`) traduzido para cláusulas `range` sobre `start_year` / `end_year`, com lógica de **sobreposição de intervalos** (um documento com início e fim corresponde se os intervalos se cruzam; um documento só com `start_year` é tratado como data pontual dentro do intervalo).

A expressão temporal é **removida do texto** da query — não é pesquisada como termo lexical.

8.2. Filtro de âmbito da visita

Padrões como "nesta visita virtual", "no tour 360", "in this tour" são detetados (`_extract_tour_scope_expression`) e convertidos num filtro booleano `in_tour=true`. Também são removidos do texto da query. Isto responde a "o que posso ver nesta visita?" restringindo aos artefactos efetivamente mapeados na visita virtual.

8.3. Filtros de seguimento

Em seguimentos, os `filters_delta` do router (ex.: `{"category": "azulejo"}`) são fundidos com os filtros herdados da sessão. Os campos suportados incluem categóricos (`category`, `creators`, `support_or_material`, `technique`, `date_or_period` ...), numéricos

(`start_year` , `end_year` , `image_count` ..., com `gte` / `lte`) e booleanos (`in_tour`).

9. Reescrita de query e *boosts*

9.1. Duas queries diferentes

Dentro de `_retrieve_context` constroem-se **duas** representações da pergunta:

- `embedding_query` — a expressão original do utilizador (após retirar temporal e âmbito da visita). É usada para o vetor semântico. Mantém-se fiel à intenção literal.
- `lexical_query` — uma versão **curta e limpa** para BM25. Por omissão é produzida por heurística (`_build_lexical_query`); se `CHAT_ENABLE_LLM_LEXICAL_QUERY=true` , o LLM extrai a expressão nominal pesquisável (`_rewrite_retrieval_query_with_llm`).

O prompt de reescrita lexical é muito restritivo: **não traduz, não troca por sinónimos, não muda singular/plural, não inclui anos nem frases de âmbito**, e preserva expressões compostas ("fatos de banho" nunca vira "banhos"; "túmulo de Fernando Pessoa" não vira "Fernando Pessoa"). Devolve apenas `{"lexical_query": "... "}`.

9.2. Boost "in_tour"

Os artefactos presentes na visita virtual recebem um *boost* configurável para subirem no ranking (sem **excluir** os restantes):

- `CHAT_IN_TOUR_BOOST` — para pesquisa de texto.
- `IMAGE_IN_TOUR_BOOST` — para pesquisa por imagem/modelo 3D.

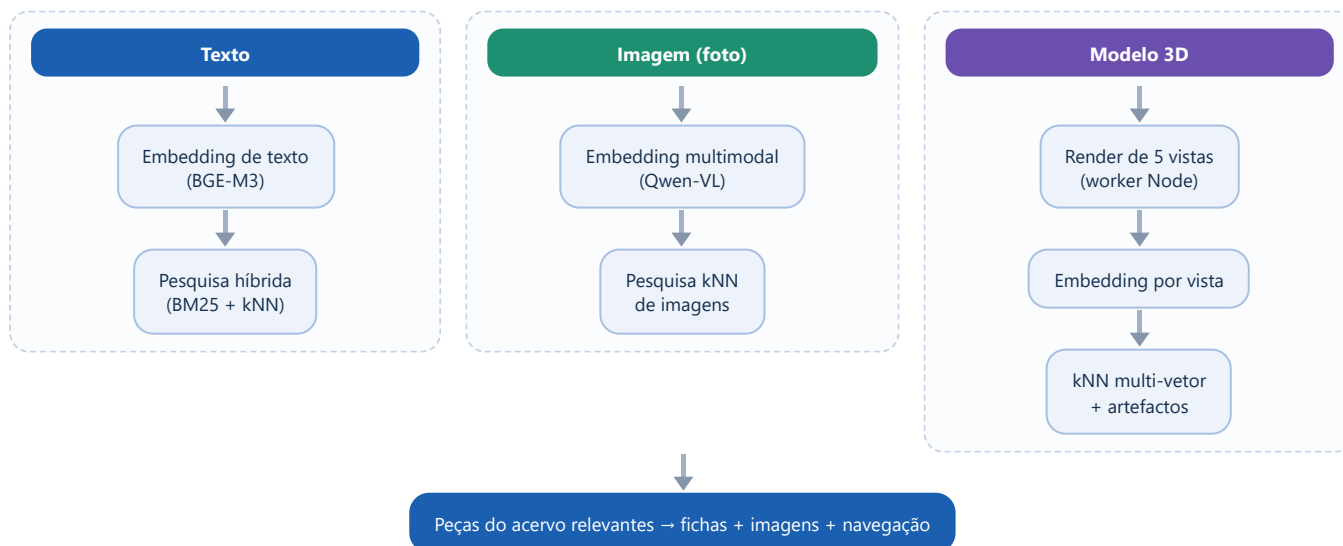
Implementado como cláusula `should` com `term: { in_tour: { value: true, boost: N } }`.

9.3. Boosts por alias de categoria/material/técnica

`app/config/retrieval_boost_aliases.json` define aliases que, quando detetados na query, acrescentam *boosts* dirigidos. Exemplo: "azulejos"/"tiles" → reforça `category: ceramica` ; "quadro"/"painting" → reforça `category: pintura` . Isto melhora a precisão sem alterar a intenção do utilizador.

10. Entradas multimodais (texto, imagem, modelo 3D)

O mesmo motor RAG aceita três tipos de "pergunta". Cada uma tem o seu ponto de entrada no `ChatService` (`handle_message` , `handle_image_message` , `handle_model_message`).



10.1. Texto

Pesquisa híbrida descrita nas secções 7–9.

10.2. Imagem

O visitante envia uma foto (upload *multipart*). Geramos um **embedding multimodal** da imagem (modelo Qwen3-VL, 2048-dim) e procuramos no índice de imagens as peças **visualmente parecidas** (`search_similar_images_page` , kNN com *boost in_tour* de imagem). A partir das imagens encontradas, resolvem-se os artefactos correspondentes. `route="image_search"` , `retrieval_mode="image_similarity"` .

10.3. Modelo 3D

O visitante envia um ficheiro `.glb` , `.gltf` ou `.obj` (validado no endpoint). O fluxo (`ModelRetrievalService`):

1. **Render** — o **Multiview Worker** (Node + Puppeteer/Chromium) renderiza **5 vistas** determinísticas do modelo. O backend faz *spawn* do worker no primeiro pedido e comunica por HTTP local. As vistas são opcionalmente persistidas em `tmp/multiview_last_views/` para inspeção.
2. **Embedding** — gera-se um embedding multimodal por vista.
3. **Pesquisa multi-vetor** — `search_similar_images_multi_page` procura imagens semelhantes a qualquer das vistas (kNN), com *boost in_tour* de imagem.
4. **Resolução de artefactos** — buscam-se os artefactos por **número de inventário** das imagens encontradas; se falhar, *fallback* por `artifact_id` .

Há **cache LRU** por *hash* do modelo (`CHAT_MODEL_CACHE_SIZE`), e existe lógica preparada para um segundo passo de vistas adicionais em caso de baixa confiança (controlada por `CHAT_MODEL_LOW_CONFIDENCE_SCORE_THRESHOLD`). `route="model_search"` , `retrieval_mode="model_multiview_similarity"` .

11. Navegação peça → ponto da visita virtual

Este é o elo distintivo do projeto: ligar **uma peça do acervo** a **um ponto físico da visita 360°**.



Estrutura de cada entrada do ficheiro de mapeamento (`poi_tours/ ... -<slug>.json`):

```
{
  "overlayId": "sprite_41F5890F_1112_FF73_41AA_15B9DDA1FFD4",
  "panoramaKey": "model_41B32FB0_110F_7293_41A3_0D87FC3F53F0",
  "inventoryIds": ["MNAz 1 Az"],
  "location": "Grande Panorama de Lisboa",
  "title": "Vista de Lisboa"
}
```

Normalização robusta: os números de inventário são passados a maiúsculas e despojados de pontuação/espacos antes do *match*; o serviço lida com **listas**, **variantes entre parênteses** e **separadores** (; , |). Se uma peça **não estiver mapeada** na visita, simplesmente não aparece o botão de navegação — a resposta de texto continua a funcionar normalmente. O índice de cada museu é carregado uma vez e mantido em memória.

12. Geração da resposta final e sanitização

A resposta final é produzida por uma chamada LLM com um prompt construído por `build_final_answer_prompt`, que injeta:

- O idioma final obrigatório (PT ou EN).
- A **precedência de fontes**: `retrieval_context` > mensagem atual > estado explícito > histórico recente > rolling summary.
- O `retrieval_context` (os cartões **visíveis** ao utilizador), como fonte primária de factos.
- A modalidade (texto/imagem/modelo) e regras específicas (ex.: numa pesquisa por imagem, o media enviado é a fonte de verdade).

Regras de consistência impostas pelo prompt:

- Nunca dizer que foram encontrados menos resultados do que os cartões visíveis.
- Descrever todos os resultados visíveis (salvo pedido explícito de subconjunto).
- **Nunca expor** identificadores internos (`artifact_id`), nomes de variáveis do prompt (`retrieval_context`, etc.) nem marcadores de contexto (`[doc_1]`, `[doc_x]`).
- Preferir o **título** (e opcionalmente a referência de inventário) ao referir uma peça.

Sanitização pós-LLM (`_sanitize_assistant_reply`): remove identificadores internos, substitui marcadores `[doc_x]` por rótulos legíveis e renúmera listas ordenadas para ficarem consecutivas. Há ainda um *guard* de idioma para garantir que a resposta sai integralmente na língua pedida.

Comportamento de falha: se o LLM estiver indisponível, devolve-se uma mensagem de *fallback* traduzida e a resposta continua a transportar os resultados de pesquisa (cartões, imagens, navegação) — a pesquisa não é perdida.

13. Paginação de resultados

Os resultados de pesquisa são **paginados**. A primeira resposta já traz a primeira página (`results_page`, `results_page_size`, `results_total`, `results_has_more`) e um `results_request_id` que identifica aquela pesquisa.

Para "ver mais resultados", o frontend chama `POST /chat/messages/results` com o `results_request_id` e o número de página. O backend tem em cache, na sessão, o pedido de retrieval e os resultados completos (`paged_results_by_request_id`), e **materializa** a página pedida sem repetir a pesquisa nem voltar a chamar o LLM de resposta (gera um texto curto de acompanhamento). Isto vale para texto, imagem e modelo 3D.

14. Modal de detalhe: autores, conjuntos, exposições, relacionados

Quando o visitante abre uma peça, o frontend pede contexto relacional **a pedido** (*lazy*), para não pesar a resposta de chat:

Endpoint	Conteúdo
<code>GET /chat/artifacts/{id}/detail-context</code>	Autores, conjuntos e exposições da peça, cada um com os outros artefactos relacionados (e <code>navigation_target</code> se existir na visita)
<code>GET /chat/artifacts/related</code>	Paginação dos artefactos de um conjunto/exposição
<code>GET /chat/artifacts/{id}/full</code>	Ficha completa de uma peça, com todas as imagens
<code>GET /chat/artifacts/by-inventory/full</code>	Ficha completa por número de inventário (usado para converter eventos da visita em referências de chat)

As entidades relacionais (autores, conjuntos, exposições) vivem em índices próprios do OpenSearch e são resolvidas pelo `OpenSearchGateway`.

15. Pesquisa entre museus (*cross-museum*)

Embora o assistente esteja embebido num museu, o utilizador pode **nomear outro museu** na pergunta (ex.: "procura trajes no Museu Nacional do Traje"). O serviço `_resolve_requested_search_museum` deteta o museu por alias (config `app/config/search_museums.json`), remove a expressão do museu da query e constrói um `search_scope` com `is_cross_museum=true`. A resposta inclui esse `search_scope`, permitindo ao frontend indicar que os resultados vêm de um museu diferente do da visita.

16. Comunicação Frontend ↔ Backend (REST + SSE)

A comunicação é **HTTP REST**, com uma camada de **streaming (SSE)** para mostrar progresso ao vivo ("a analisar pedido...", "a procurar no acervo...", "encontrei N peças...").



16.1. Endpoints (todos sob `/api/v1/chat`)

Endpoint	Função
<code>GET /health</code>	"Aquecer" a sessão e confirmar que o backend está vivo
<code>POST /messages</code> · <code>/messages/stream</code>	Pergunta por texto (simples · com progresso SSE)
<code>POST /messages/image</code> · <code>/image/stream</code>	Pergunta com imagem (upload multipart)
<code>POST /messages/model</code> · <code>/model/stream</code>	Pergunta com modelo 3D (upload multipart)
<code>POST /messages/regenerate</code> · <code>/regenerate/stream</code>	Voltar a gerar a última resposta
<code>POST /messages/results</code>	Paginação — "ver mais resultados"
<code>GET /artifacts/{id}/detail-context</code>	Contexto relacional (modal, <i>lazy</i>)
<code>GET /artifacts/related</code>	Paginação de relacionados (modal)
<code>GET /artifacts/{id}/full</code>	Ficha completa por <code>artifact_id</code>
<code>GET /artifacts/by-inventory/full</code>	Ficha completa por número de inventário
<code>GET /images/{ref}</code>	Servir ficheiros de imagem das peças

Há ainda, fora de `/chat` : `POST /api/v1/logs/events` (eventos de interação do frontend).

16.2. Princípios de desenho

- **Estado no backend** — o frontend só guarda o `conversationId` ; histórico, filtros ativos e últimos resultados ficam no servidor (permite seguimento natural).
- **Streaming opcional** — se o frontend passa um `callback` de progresso, usa a versão `/stream` (SSE); caso contrário, faz um pedido simples.

- **Tradução de formato** — o backend responde em `snake_case` ; o `chatApi.ts` normaliza tudo para `camelCase` e tipos do frontend, isolando a UI de mudanças no backend.
- **Carregamento preguiçoso** — detalhes pesados (contexto relacional, ficha completa, páginas extra) só são pedidos quando o visitante interage.
- **Metadata de correlação** — o frontend pode anexar `session_id` , `participant_id` , `task_id` e a **peça selecionada** (`selected_artifact`) no campo `metadata` , propagados para o backend e para os logs.

17. Comunicação Frontend ↔ Visita Virtual (postMessage)

A visita virtual é uma aplicação **externa** carregada num `<iframe>` . O assistente e a visita comunicam por `postMessage` (entre janelas do browser) — não há servidor pelo meio. A comunicação é **bidirecional**.

17.1. Assistente → Visita

Mensagem	Conteúdo	Quando
<code>patrimonio360:tour-context</code>	<code>{ museumSlug }</code>	Ao carregar a visita (sincroniza o contexto do museu)
<code>navigateToArtifact</code>	<code>{ overlayId, panoramaKey, inventoryId, requestId }</code>	Quando o visitante clica em "Ver na visita" num resultado

17.2. Visita → Assistente

A visita reporta eventos que o `TourAssistantEmbed` escuta:

Mensagem (<code>event.data.type</code>)	Significado
<code>selectedArtifacts</code>	O utilizador abriu/selecionou uma peça dentro da visita (traz inventário, título, localização)
<code>deselectedArtifacts</code>	Fechou a seleção
<code>tour_event</code> (com <code>event_type</code>)	Eventos de navegação: <code>tour_location_changed</code> , <code>artifact_info_opened</code> , <code>artifact_info_closed</code> , <code>navigation_completed</code> , etc.

Isto permite **correlacionar** o que acontece na visita com a conversa: por exemplo, quando o assistente envia `navigateToArtifact` e a visita responde com `navigation_completed` , o frontend regista o sucesso e associa o evento à query original.



18. Sessão, estado e memória de conversa

O estado de conversa vive em memória no backend (`ChatSessionStore`), indexado por `conversation_id` , com **eviction por TTL** (`CHAT_SESSION_TTL_SECONDS` , 3600s por omissão). Cada `ChatSessionState` guarda:

- `language` , `intent` atual.
- `filters` e `sort` ativos (para seguimentos).
- `history` (janela das últimas N mensagens — `CHAT_HISTORY_WINDOW` , 8 por omissão).
- `rolling_summary` — resumo contínuo da conversa (máx. `CHAT_ROLLING_SUMMARY_MAX_CHARS` , 600).
- `last_result_ids` e os **resultados paginados em cache** (`paged_results_by_request_id`) para servir "ver mais" sem repesquisar.

Se o `conversation_id` mudar de museu, o estado ligado ao museu é reiniciado (mas o id mantém-se). O **rolling summary** é atualizado a cada turno e serve apenas como contexto auxiliar — nunca introduz factos novos na resposta.

19. Internacionalização (PT/EN)

Todo o texto voltado ao utilizador (estados de progresso, erros, mensagens *default* de upload, *guards* de idioma e rótulos do sanitizador) é traduzido via `chat_i18n.translate` em **português** e **inglês**, com suporte a pluralização (ex.: "Encontrado 1 artefacto" vs "Encontrados 5 artefactos"). O idioma é resolvido do pedido (`language`) e fixado na sessão. Os prompts do router, da reescrita e da resposta final também têm variantes PT/EN.

20. Observabilidade: logging estruturado de queries e interações

O sistema produz **dois fluxos de log JSONL** (um objeto JSON por linha), pensados para avaliação offline e análise (papers). O serviço `QueryLogger` é **fail-soft**: nunca quebra a resposta de chat; em caso de falha, emite um *warning* e continua. Não regista chaves de API, *tokens*, cabeçalhos nem segredos.

20.1. `backend_query` (um evento por pedido de chat)

Escrito em `logs/evaluation/backend_queries.jsonl` (configurável via `QUERY_LOG_PATH`). Gerado nos três pontos de entrada (texto, imagem, modelo 3D), com um `query_id` único por pedido. Inclui IDs de correlação, a query em várias formas, modo/rota de retrieval, filtros e *boosts* aplicados, artefactos recuperados e mostrados, alvos de navegação, latências (retrieval / LLM / total), estado e — no fundo do evento — ***o dump exato da query enviada ao OpenSearch*** (campo `opensearch_query` , incluindo o vetor de embedding completo nas pesquisas de texto). O esquema completo está no Apêndice B.

20.2. `frontend_events` (interações do utilizador)

Escrito em `logs/evaluation/frontend_events.jsonl` (`FRONTEND_EVENT_LOG_PATH`), recebido via `POST /api/v1/logs/events` . O frontend envia eventos (com `sendBeacon` / `fetch`) como `assistant_opened` , `message_sent` , `answer_received` , `artifact_card_opened` , `see_in_tour_clicked` , `navigation_command_sent` , `navigation_completed` , `tour_location_changed` , `task_started` , `task_completed` , `feedback_clicked` , `error_shown` , entre outros. Cada evento transporta `session_id` , `conversation_id` , `query_id` , `participant_id` , `task_id` , `tour_id` e `language` , permitindo reconstruir a jornada completa do visitante e cruzá-la com os `backend_query` .

Privacidade: os logs não são expostos na UI normal do frontend; servem avaliação e investigação. Não contêm dados pessoais desnecessários nem segredos.

21. Configuração (settings/env)

A configuração vive em `app/core/config.py` (Pydantic `BaseSettings` , lê `backend/.env`). Grupos principais:

Grupo	Exemplos de chaves
Ambiente / CORS	<code>APP_ENV</code> , <code>API_PREFIX</code> , <code>CORS_ALLOW_ORIGINS</code>
OpenSearch	<code>OPENSEARCH_HOST/PORT/SCHEME</code> , índices <code>OPENSEARCH_INDEX_ARTIFACT/IMAGE/MUSEUM/ ...</code>

Grupo	Exemplos de chaves
Embeddings	<code>QWEN_TEXT_EMBEDDING_MODEL_ID</code> , <code>QWEN_MULTIMODAL_EMBEDDING_MODEL_ID</code> , <code>USE_OPENROUTER_BGE_M3</code> , dimensões
LLM	<code>LLM_PROVIDER</code> , <code>LLM_BASE_URL</code> , <code>LLM_MODEL</code> , <code>LLM_TEMPERATURE_TEXT/JSON</code> , <code>LLM_TIMEOUT_SECONDS</code>
Chat / retrieval	<code>CHAT_ENABLE_RAG</code> , <code>CHAT_ENABLE_LLM_LEXICAL_QUERY</code> , <code>CHAT_USE_QUERY_EMBEDDINGS</code> , <code>CHAT_RETRIEVAL_CANDIDATES</code> , <code>CHAT_RETRIEVAL_TOP_K</code> , <code>CHAT_IN_TOUR_BOOST</code>
Imagem / 3D	<code>CHAT_IMAGE_RETRIEVAL_TOP_K</code> , <code>IMAGE_IN_TOUR_BOOST</code> , <code>CHAT_MODEL_TOTAL_VIEWS</code> , <code>CHAT_MODEL_CACHE_SIZE</code>
Multiview worker	<code>MULTIVIEW_WORKER_HOST/PORT</code> , <code>MULTIVIEW_RENDER_*</code>
Caminhos	<code>IMAGE_ASSET_ROOT</code> , <code>POI TOURS_DIR</code> , <code>MULTIVIEW_LAST_VIEWS_DIR</code>
Logging	<code>QUERY_LOG_ENABLED</code> , <code>QUERY_LOG_PATH</code> , <code>FRONTEND_EVENT_LOG_PATH</code>

Os serviços são *singletons* construídos por *factories* com `@lru_cache` (`get_chat_service`, `get_settings`, `get_opensearch_gateway`, `get_query_logger`, ...), injetados nas rotas via `Depends( ... )` do FastAPI.

22. Modelo de dados (artefacto e entidades)

O objeto central devolvido ao frontend é o `ArtifactResult`. Campos principais:

- **Identificação:** `artifact_id`, `inventory_number`, `tipo_inventario`, `title`.
- **Museu:** `museum_id`, `museum`.
- **Classificação:** `category`, `super_category`, `support_or_material`, `technique`.
- **Autoria:** `creator` (legado), `creators[]`, `creator_ids[]`.
- **Cronologia:** `date_or_period`, `start_year`, `end_year`.
- **Texto:** `description`, `origin_history`, `incorporation`, `production_center`, `bibliography`.
- **Relações:** `sets[]`, `set_ids[]`, `exhibitions[]`, `exhibition_ids[]`, contagens.
- **Visita:** `in_tour` (booleano).
- **Imagens:** `images[]` (`ArtifactImageResult`: `image_id`, `local_path`, `source_url`, `caption`, `alt_text`, `image_order`).

Entidades relacionais para o modal: `AuthorEntity`, `SetEntityWithArtifacts`, `ExhibitionEntityWithArtifacts` (cada uma com a sua lista de `RelatedArtifactItem`, incluindo `navigation_target`). Resultados de imagem usam `ImageMatchResult` e a navegação usa `TourNavigationTarget` (`overlay_id`, `panorama_key`, `inventory_id`, `location`, `title`).

23. Exemplos de todos os tipos de queries

Esta secção percorre **todos os tipos de pergunta** que o sistema reconhece, mostrando a entrada, como é classificada e o que acontece.

23.1. Pergunta factual sobre uma peça (RAG, texto)

- **Utilizador:** "Fala-me sobre o traje de criança."
- **Router:** `mode=rag`, `intent=search`, `needs_retrieval=true`, sem filtros.
- **Retrieval:** embedding de "traje de criança" + BM25 (`lexical_query` ≈ "traje criança"); pesquisa híbrida no museu atual.
- **Resposta:** texto descritivo baseado nas peças encontradas + cartões de artefacto; botão "Ver na visita" nas peças mapeadas.

23.2. Pergunta aberta / exploratória (RAG, texto)

- **Utilizador:** "Que peças há sobre arte sacra?"
- **Router:** `mode=rag`, `intent=search` / `overview`.
- **Retrieval:** híbrido; o *boost* de alias pode reforçar categorias relevantes.
- **Resposta:** lista dos resultados visíveis (primeira página), consistente com os cartões.

23.3. Pergunta quantitativa (RAG, sem executor analítico)

- **Utilizador:** "Quantos azulejos do século XVIII há nesta visita?"

- **Router:** `mode=rag`. A expressão "nesta visita" ativa o filtro `in_tour=true`; "século XVIII" é interpretado como intervalo temporal (filtro de metadata).
- **Nota:** não há executor analítico de contagem exata nesta versão; a resposta é redigida com base nos resultados recuperados (e no `results_total` estimado), não por um agregador numérico dedicado.

23.4. Lookup direto por número de inventário (RAG, atalho)

- **Utilizador:** "Mostra-me a peça MNAz 1 Az" ou "número de inventário 1234".
- **Deteção:** `_extract_inventory_candidates` reconhece prefixos de museu (`mnaz`, `mnt`, `mj`, `mnsr`, ...) e marcadores ("nº de inventário", "referência").
- **Retrieval:** atalho `_retrieve_inventory_context` — procura diretamente por candidatos de inventário, sem passar pela pesquisa híbrida (`retrieval_mode` interno `kind="inventory"`).
- **Resposta:** a ficha da peça exata + navegação.

23.5. Pergunta com filtro temporal explícito (RAG, texto)

- **Utilizador:** "Azulejos entre 1700 e 1750."
- **Retrieval:** "entre 1700 e 1750" → filtro `range` sobre `start_year` / `end_year` (sobreposição de intervalos). O texto pesquisado fica só "azulejos".

23.6. Pergunta com período histórico nomeado (RAG, texto)

- **Utilizador:** "O que há do período pombalino?"
- **Retrieval:** "período pombalino" → intervalo `hardcoded` 1750–1777 (filtro temporal).

23.7. Pergunta com âmbito da visita (RAG, texto)

- **Utilizador:** "O que posso ver nesta visita virtual?"
- **Política de contexto:** trata como **nova pesquisa autónoma**; ativa `in_tour=true`.
- **Resposta:** apenas peças efetivamente mapeadas na visita 360°.

23.8. Pesquisa entre museus (RAG, cross-museum)

- **Utilizador (numa visita do MNAZ):** "Procura trajes no Museu Nacional do Traje."
- **Deteção:** alias "Museu Nacional do Traje" → `search_scope` para MNT com `is_cross_museum=true`; a expressão do museu é removida da query.
- **Resposta:** resultados do MNT, com indicação de que vêm de outro museu.

23.9. Seguimento referencial — refina a query (RAG, follow-up)

- **Contexto:** pesquisa anterior por "azulejos".
- **Utilizador:** "E só os de iconografia mariana?" / "Só os do século XVIII."
- **Política:** seguimento; `use_history_for_query=true`, `carry_filters=true`.
- **Retrieval:** mantém a pesquisa anterior e **acrescenta** o novo filtro/termo.

23.10. Seguimento referencial — só resposta (sem repesquisa)

- **Utilizador:** "Explica melhor o primeiro." / "Detalha essa peça."
- **Política:** `use_history_for_answer=true`, `use_history_for_query=false`.
- **Comportamento:** não repesquisa; reusa o contexto e redige com mais detalhe.

23.11. Peça selecionada como âncora (RAG, contexto de seleção)

- **Contexto:** o visitante selecionou uma peça (na visita ou num cartão); o frontend envia `selected_artifact` em `metadata`.
- **Utilizador:** "Há mais peças parecidas com esta?"
- **Comportamento:** o backend ancora a pesquisa na peça selecionada (similaridade / contexto da peça) para responder de forma relevante.

23.12. Conversa / saudação (LLM-only)

- **Utilizador:** "Olá!" / "Obrigado, até já."
- **Router:** `mode=llm_only` (o `fallback` heurístico também trata saudações).
- **Comportamento:** não vai ao acervo; responde só com o contexto da conversa.

23.13. Pesquisa por imagem (RAG multimodal)

- **Utilizador:** envia uma foto de uma peça (com ou sem texto).
- **Fluxo:** embedding multimodal da imagem → kNN no índice de imagens → artefactos correspondentes. `route="image_search"`.
- **Resposta:** "a peça mais provável" + visualmente semelhantes + navegação.

23.14. Pesquisa por modelo 3D (RAG multimodal)

- **Utilizador:** envia um `.glb` / `.gltf` / `.obj`.
- **Fluxo:** render de 5 vistas (worker) → embedding por vista → kNN multi-vetor de imagens → artefactos por inventário. `route="model_search"`. O primeiro pedido pode demorar mais (arranque do worker).

23.15. Paginação ("ver mais resultados")

- **Utilizador:** clica "ver mais".
- **Fluxo:** `POST /messages/results` com `results_request_id` → materializa a página seguinte a partir da cache da sessão, **sem** repesquisar nem chamar o LLM de resposta.

23.16. Regenerar resposta

- **Utilizador:** pede para gerar de novo a última resposta.
- **Fluxo:** `POST /messages/regenerate` — reexecuta o último turno do utilizador (gera um novo `query_id`).

23.17. Detalhe de uma peça (modal, lazy)

- **Utilizador:** abre uma peça nos resultados.
- **Fluxo:** `GET /artifacts/{id}/detail-context` → autores, conjuntos, exposições e artefactos relacionados (carregados só nesse momento).

23.18. Pergunta sem dados / fora de domínio

- **Utilizador:** "Qual é a capital da Austrália?"
- **Comportamento:** o router tende a `llm_only`; se cair em RAG sem resultados, o prompt obriga a responder com cautela e a indicar que faltam dados factuais do acervo.

24. Apêndice A — Esquema do Router (JSON)

O router devolve estritamente este objeto:

```
{
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "mode": { "type": "string", "enum": ["rag", "llm_only"] },
    "intent": { "type": "string", "enum": ["overview", "search", "refine", "fallback"] },
    "is_follow_up": { "type": "boolean" },
    "use_history_for_query": { "type": "boolean" },
    "use_history_for_answer": { "type": "boolean" },
    "carry_filters": { "type": "boolean" },
    "carry_sort": { "type": "boolean" },
    "rewritten_query": { "type": "string" },
    "needs_retrieval": { "type": "boolean" },
    "reason": { "type": "string" },
    "filters_delta": { "type": "object" },
    "sort_delta": { "type": "object" }
  },
  "required": ["mode", "intent", "is_follow_up", "use_history_for_query",
    "use_history_for_answer", "carry_filters", "carry_sort",
    "rewritten_query", "needs_retrieval", "reason",
    "filters_delta", "sort_delta"]
}
```

Exemplo de saída para *"E só os de iconografia mariana?"* (seguimento de "azulejos"):

```
{
  "mode": "rag",
  "intent": "refine",
  "is_follow_up": true,
  "use_history_for_query": true,
  "use_history_for_answer": true,
  "carry_filters": true,
  "carry_sort": true,
  "rewritten_query": "azulejos com iconografia mariana",
  "needs_retrieval": true,
  "reason": "refine sobre pesquisa anterior",
  "filters_delta": { "category": "azulejo" },
  "sort_delta": {}
}
```

25. Apêndice B — Esquema do evento `backend_query`

Um objeto por linha em `logs/evaluation/backend_queries.jsonl`:


```

{
  "event_type": "backend_query",
  "timestamp": "2026-06-18T10:15:30.123456+00:00",
  "session_id": null,
  "conversation_id": "conv-abc",
  "query_id": "fle2d3c4-...",
  "participant_id": null,
  "task_id": null,
  "tour_id": "mnaz",
  "museum_slug": "mnaz",
  "language": "pt",
  "raw_query": "azulejos do século XVIII nesta visita",
  "resolved_query": "azulejos do século XVIII",
  "lexical_query": "azulejos",
  "embedding_query": "azulejos",
  "route": "rag",
  "retrieval_mode": "hybrid-text",
  "filters_applied": {
    "filters": { "in_tour": true },
    "sort": {},
    "temporal_query": { "start_year": 1700, "end_year": 1799 },
    "tour_scope": { "in_tour": true }
  },
  "boosts_applied": { "in_tour_boost": 5 },
  "retrieved_artifacts": [
    { "artifact_id": "a1", "inventory_number": "MNAz 1 Az", "title": "...", "score": 12.34, "source": "hybrid" }
  ],
  "shown_artifacts": [
    { "artifact_id": "a1", "inventory_number": "MNAz 1 Az", "title": "..." }
  ],
  "navigation_targets": [
    { "artifact_id": "a1", "inventory_number": "MNAz 1 Az", "has_navigation_target": true,
      "panorama_id": "model...", "overlay_id": "sprite..." }
  ],
  "latency_retrieval_ms": 120.4,
  "latency_llm_ms": 800.1,
  "latency_total_ms": 950.7,
  "status": "ok",
  "error": null,
  "opensearch_query": {
    "query": { "hybrid": { "pagination_depth": 150, "queries": [
      { "knn": { "text_embedding": { "vector": [0.0123, -0.0456, "...1024 floats..."], "k": 1000 } } },
      { "bool": { "must": [ { "bool": { "should": [ "...multi_match BM25..." ] } } ] } }
    ] } },
    "size": 15
  }
}

```

Para `image_search` e `model_search`, `lexical_query` / `embedding_query` são `null`, `boosts_applied` usa `image_in_tour_boost`, e `opensearch_query` reflete a query de imagem.

26. Apêndice C — Notas de implementação e componentes não ativos

- **Planeador analítico removido:** uma versão anterior tinha um *query planner* analítico (`app/query_planning/`). Foi **removido**; não há executor de contagens/agregações exatas. Perguntas quantitativas são respondidas pela via RAG normal (secção 23.3).
- **Reranker (não ativo):** existe `services/reranker.py` (reranker pairwise estilo Qwen), mas **não está ligado** ao pipeline e depende de settings que não estão em `config.py` (`RERANKER_*`, `CHAT_ENABLE_RERANKING`). É código preparatório/experimental.

- **Pré-aquecimento (warmup):** `warmup.py` permite pré-carregar OpenSearch, embeddings e worker; está **desligado por omissão** (`CHAT_PREWARM_ON_STARTUP=false`) e não é chamado no arranque atual de `main.py` .
 - **Worker 3D sob demanda:** o Multiview Worker não precisa de ser arrancado manualmente; o backend faz *spawn* no primeiro pedido de pesquisa 3D, desde que o Node.js e as dependências de `backend/multiview_worker/` estejam instalados.
-

Documento gerado a partir da análise direta do código-fonte do repositório `patrimonio360_assistant` (backend FastAPI + frontend React).